

Recommender System

Matylda Pańczyk, Joanna Pokora

1 Introduction

The objective of this project is to develop a recommendation system for movie ratings using the following algorithms: Non-negative Matrix Factorization (NMF), two versions of Singular Value Decomposition (SVD1 and SVD2) and Stochastic Gradient Descent (SGD). Our approach further extends the basic models by incorporating modifications of NMF and SVD2, as well as two alternative loss functions for SGD. All models will be evaluated using the root-mean-square error (RMSE) and the best approach will be selected.

2 Data

The training dataset used in this study comes from the `ratings.csv` file, included in the `ml-latest-small.zip` available on the GroupLens website [1]. The dataset contains approximately 100 000 ratings with around 600 users and 9 000 movies. The columns of interest are

- `userId` - unique identifier for each user,
- `movieId` - unique identifier for each movie,
- `rating` (float) - given by user score between 0.0 and 5.0.

```
userId,movieId,rating
1,1,4.0
1,3,4.0
1,6,4.0
1,47,5.0
1,50,5.0
...
```

For testing, we use data from the files `sample_test.csv` and `sample_test_with_ratings.csv`, provided in the `sample_project1` folder on the course website.

2.1 Matrix representation

We transform the `rating` column from `ratings.csv` into a matrix form. Let us denote it as a matrix Z of size $n \times d$, where n is the number of users and d is the number of movies. First, we fill it with values from the dataset. At this point, Z is sparse, as many user-movie pairs are not present in the initial data. Since the NMF and SVD1 methods require a complete matrix Z , we impute the missing values with the average rating for each movie, that is, the column-wise mean, before training these models. We also create dictionaries mapping the user and movie IDs to the corresponding rows and columns of Z , respectively, starting from 0.

3 Methods

We consider four different approaches to predicting ratings. In this section, we provide their detailed descriptions.

3.1 Non-negative Matrix Factorization (NMF)

Given the matrix Z of size $n \times d$ with non-negative elements, the objective of NMF is to find two matrices, W and H , of sizes $n \times r$ and $r \times d$, respectively, also with non-negative elements, such that

$$Z \approx WH.$$

This model requires a complete matrix Z , therefore we use the imputed data for training.

The NMF implementation used in this study comes from the `sklearn` package [2] and, with our settings, it looks for the matrices minimizing

$$L(W, H) = \frac{1}{2} \|Z - WH\|_F^2,$$

where $\|X\|_F^2 = \sum_{i,j} x_{i,j}^2$ denotes the squared Frobenius norm.

In addition, we considered several initialization methods. The best one in terms of RMSE on the test was `nndsvd` (Nonnegative Double Singular Value Decomposition), with RMSE = 0.7. Similar results were obtained with `init=random` (RMSE = 0.714). The fastest, but also the least accurate, method was `nndsvda`, with RMSE = 0.779. Therefore, in the final algorithm we used the `nndsvd` initialization.

3.2 Singular Value Decomposition - first algorithm (SVD1)

The singular value decomposition of the matrix $Z \in \mathbb{R}^{n \times d}$ is given by

$$Z = U\Lambda V^T,$$

where

- Λ is a diagonal matrix of size $n \times d$ with square roots of the nonnegative eigenvalues of ZZ^T , called the singular values,
- U is an $n \times n$ orthonormal matrix whose columns are orthonormal eigenvectors of ZZ^T ,
- V is an $d \times d$ orthonormal matrix whose columns are orthonormal eigenvectors of $Z^T Z$.

In the project, we use the truncated SVD, defined as

$$Z \approx U_r \Lambda_r V_r^T,$$

where U_r , Λ_r and V_r^T are matrices of sizes $n \times r$, $r \times r$ and $r \times d$, respectively. To maintain consistency with the NMF, we set $W = U_r \Lambda_r$ and $H = V_r^T$, which yields $Z \approx WH$.

3.3 Singular Value Decomposition - second algorithm (SVD2)

In the second algorithm, we replace missing values in matrix Z with zeros. Then, we perform an iterative approximation process. In each iteration, the truncated singular value decomposition is computed:

$$Z^{(n)} \approx U_r \Lambda_r V_r^T,$$

where $U_r \Lambda_r = W$ and $V_r^T = H$.

In the algorithm, we update the current matrix $Z^{(n+1)}$ by keeping the original ratings where $z_{ij} > 0$, $i \in \{1, \dots, n\}$, $j \in \{1, \dots, d\}$, and filling the missing entries with values from the predicted matrix $Z^{(n)}$. We assess convergence by calculating the absolute difference between RMSE values of known ratings from SVD_r approximation of $Z^{(n+1)}$ and $Z^{(n)}$, from the current and previous iterations. If this difference is smaller than the predefined tolerance level (10^{-6}), then we have reached a fixed point. Otherwise, we set the predicted matrix as the current one and repeat the process. Finally, the model returns the matrices W and H as the solution.

Additionally, we set the maximum number of iterations to 20 during cross-validation and to 1000 when the model was trained on the full data matrix.

3.4 Stochastic Gradient Descent (SGD)

In this method we do not fill missing data. For a given Z of size $n \times d$ we seek matrices W (size $n \times r$) and H (size $r \times d$) minimizing the given loss function:

$$\arg \min_{W, H} \sum_{(i,j): z_{ij} \neq ?} (z_{ij} - \mathbf{w}_i^T \mathbf{h}_j)^2,$$

where $\mathbf{h}_j$ is the j -th column and $\mathbf{w}_i$ is the i -th row of W .

In the implementation, we create a binary mask that allows model to learn only from the ratings actually provided. We utilized the PyTorch library and tensors, which enabled automatic gradient computation for W and H matrices.

During the model selection process, we compared two optimizers: SGD and Adam. It turned out that the Adam optimizer achieved better accuracy - it minimizes the RMSE more effectively than SGD. Consequently, we chose Adam optimizer with learning rate 0.01 as the default for the final implementation.

3.5 Best r

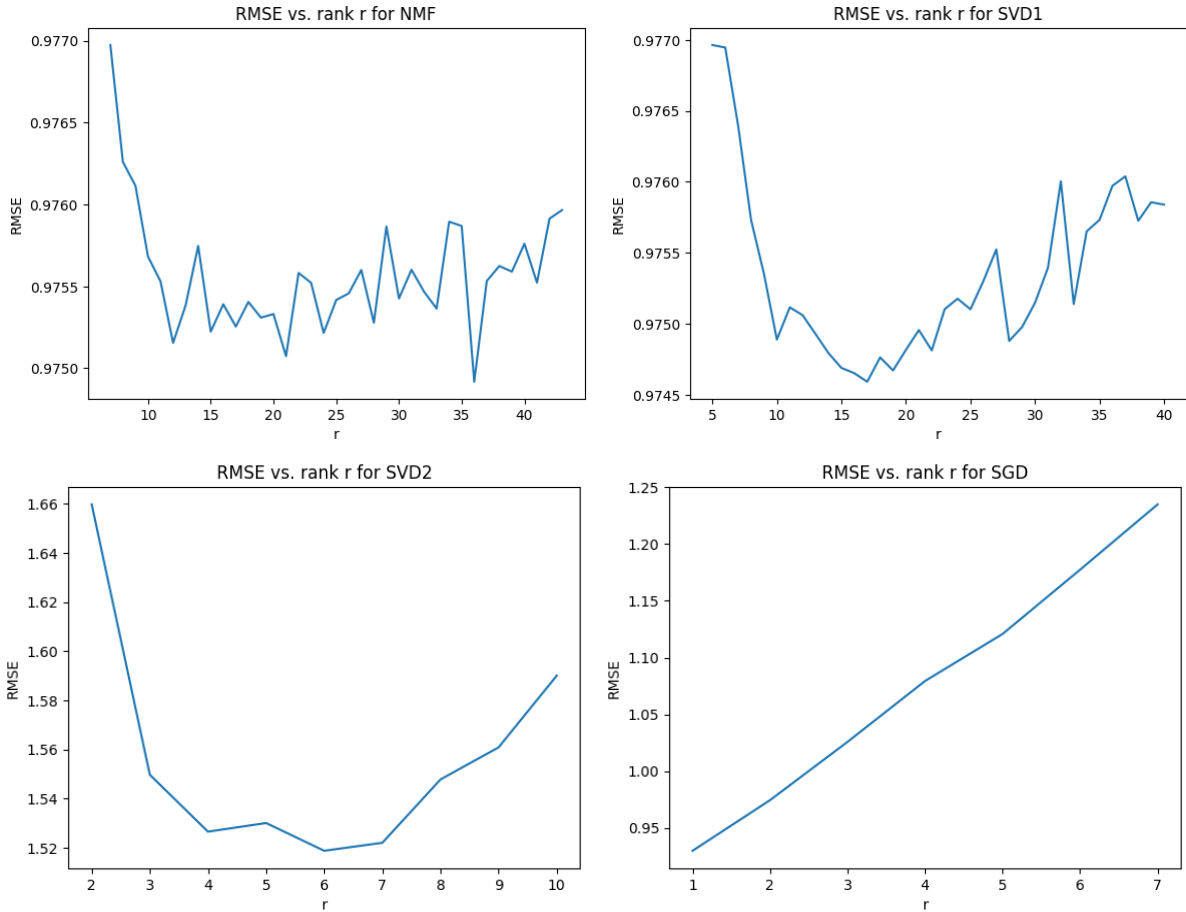
For each of the previously described methods, the parameter r - representing the rank of the matrix approximation - plays a crucial role. To ensure optimal model performance, we conducted a 5-fold cross-validation process for each algorithm to select the best value for this parameter.

First, we partitioned the training data into five folds. Then we iterated over them so that each subset served exactly once as the validation set, and calculated RMSE for each fold. This allowed us to assess the model's performance on unseen data and select the r that minimized RMSE.

During the preliminary phase, we performed tests for multiple values of r . Based on these initial results, we identified convergence trends and narrowed the search to intervals where the error was lowest, significantly reducing the computation time required for each method. The following plots illustrate the relationship between the RMSE and the rank r for each algorithm.

Method	Min r	Max r	Best r
NMF	7	43	36
SVD1	5	40	17
SVD2	2	10	6
SGD	1	7	1

Table 1: Ranges of parameter r used in cross-validation for each algorithm.



4 Best algorithm

In addition to the mandatory models, we tried altering them to find the best-performing approach. Unfortunately, the RMSE values of the new models on the test set were higher than that of SVD2, which we therefore selected as the BEST approach. Nevertheless, we decided to include these algorithms in the project as NMF_V2, SVD2_V2 and SGD_REG.

1. The NMF_V2 method is a modification of NMF, analogous to the SVD2 approach. We perform NFM twice - after the first run, we replace the missing entries in the initial matrix with the obtained predictions. Then, we apply the regular NMF procedure with the rank reduced to rounded $r/2$.
2. In the method SVD2_V2, we fill in the missing entries with column and global means instead of zeros, thus we first perform the imputation as in NMF or SVD1. Then, we apply the regular SVD2 algorithm.
3. In the SGD_REG model, we added regularization to the previous loss function:

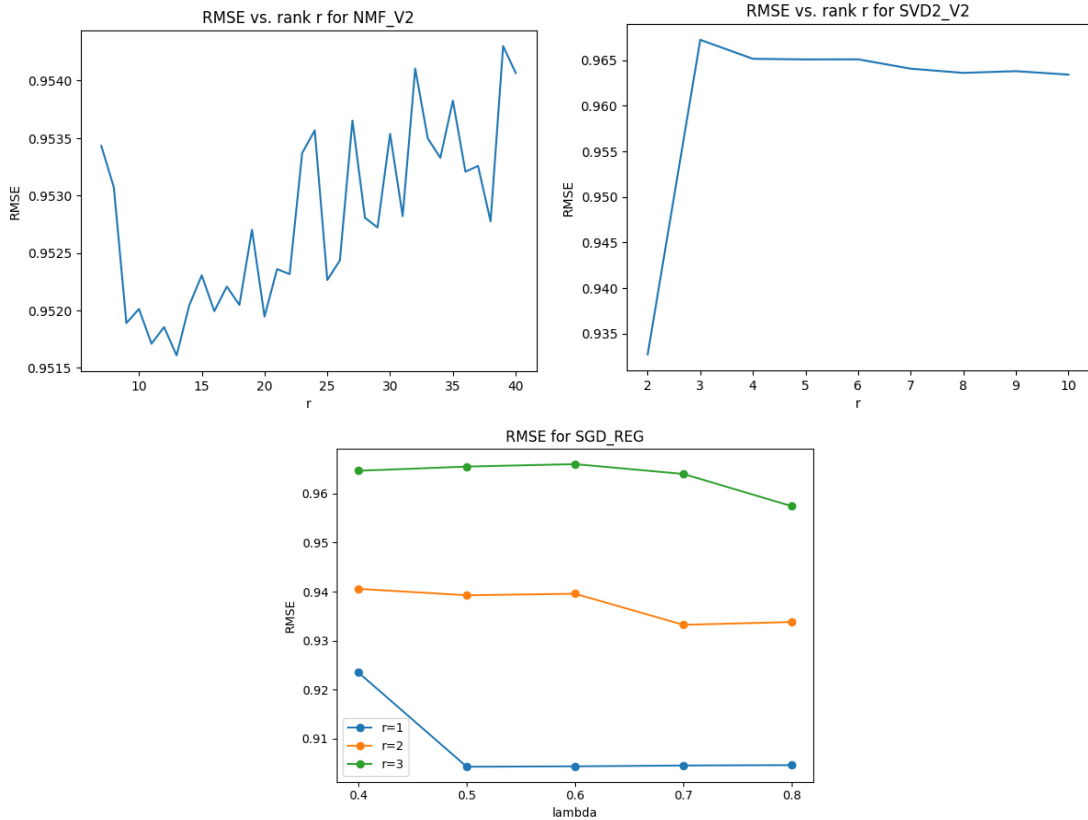
$$\arg \min_{W, H} \sum_{(i,j): z_{ij} \neq '?'} (z_{ij} - \mathbf{w}_i^T \mathbf{h}_j)^2 + \lambda (\|\mathbf{w}_i\|^2 + \|\mathbf{h}_j\|^2),$$

where λ is the parameter controlling the strength of the regularization.

The table below contains all considered values of r , together with the best ranks estimated using cross-validation. Additionally, the plots depict the RMSE for each value of r and, in the case of SGD_REG, for all values of λ .

Method	Min r	Max r	Best r
NMF_V2	5	40	17
SVD2_V2	2	10	6
SGD_REG	1	3	1

Table 2: Ranges of parameter r used in cross-validation for each algorithm.



5 Results

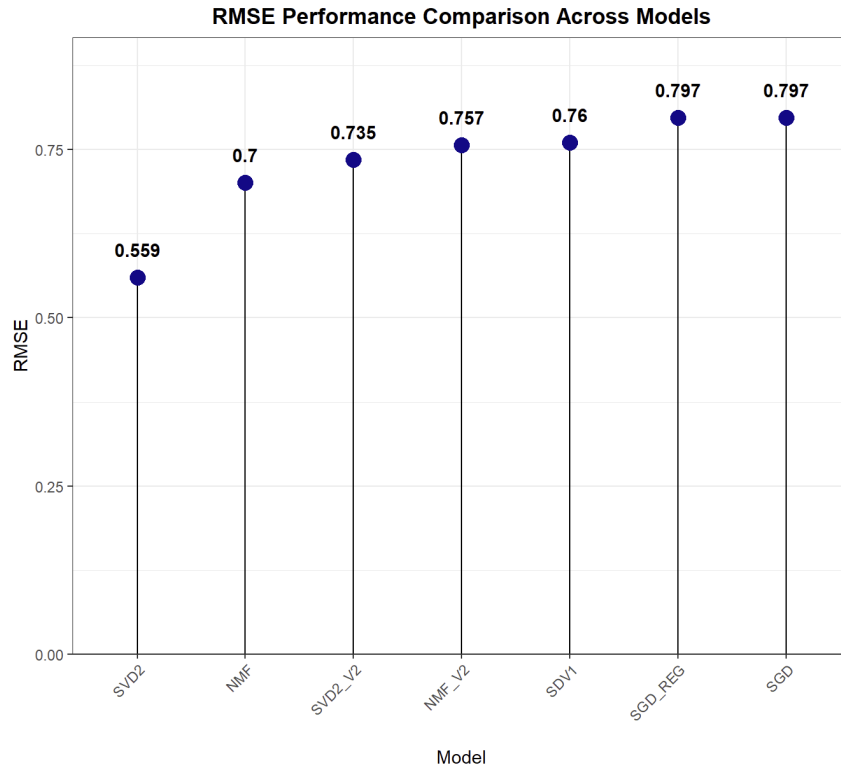
All models were evaluated using the root-mean square error

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,m) \in \mathcal{T}} (\mathbf{Z}'[u, m] - \mathbf{T}[u, m])^2},$$

where $\mathcal{T}$ denotes the test set, $\mathbf{Z}'[u, m]$ is the predicted rating of user u for movie m and $\mathbf{T}[u, m]$ is the true rating.

All RMSE values obtained in the study are presented in the table below.

	NMF	SVD1	SVD2	SGD	NMF_V2	SVD2_V2	SGD_REG
RMSE	0.700	0.760	0.559	0.797	0.757	0.735	0.797



The most accurate method was SVD2. The second best approach was NMF, then SVD2_V2 and so on.

References

- [1] *GroupLens archive*. <https://grouplens.org/datasets/movielens/>. Accessed: 2026-04-23.
- [2] *sklearn - NMF method*. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.NMF.html>. Accessed: 2026-04-23.